

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчет об исследовательском проекте на тему:
«Методы анализа временных рядов и обнаружения изменений»

Выполнил:

студент группы БПМИ244
Крутиков Никита

Принял руководитель проекта:

Лукьянченко Петр Павлович
Заведующий проектно-учебной лабораторией
«Искусственный интеллект в математических финансах»

Москва 2026

Содержание

1	Введение и постановка задачи	2
1.1	Актуальность проблемы	2
1.2	Цель, новизна и задачи	2
2	Обзор литературы и мотивация	3
2.1	Общие сведения о CPD	3
2.2	Категории методов	4
2.3	Сравнительные исследования	4
2.4	Что следует из обзора	4
3	Обзор методов CPD	5
3.1	Детали архитектуры и обучения ML-моделей	6
3.2	Сравнение версий: v1 и v2	8
4	Синтетические данные на основе SDE	9
4.1	Модель генерации	9
4.2	Параметры и их влияние	9
5	Протокол экспериментов	10
5.1	Метрики качества	11
5.2	Разделение данных	12
5.3	Сопоставление и сценарии	12
5.4	Устройство пайплайна	13
5.5	Исходные гипотезы	13
6	Статистический анализ SDE-процесса	14
6.1	H1: Перелеты через границу – half-normal	14
6.2	H2: Форма распределения временных расстояний между CP	15
6.3	H3: Пуассоновость числа CP	16
6.4	Сводка гипотез	17
7	Экспериментальные результаты	18
7.1	Сравнение методов при $D = 1.0$ (рабочий режим)	18
7.2	Сравнение при $D = 0.5$ (простой режим)	19
7.3	Влияние типа шума и уровня диффузии	20
7.4	Точность локализации	21
7.5	Сводка по сценариям	22
8	Интерпретация и обсуждение	22
8.1	Почему разные методы побеждают в разных режимах	22
8.2	Режимный анализ	23
8.3	Анализ ошибок	23
8.4	Вычислительная сложность	24
8.5	Ограничения	24
9	Заключение	24

1 Введение и постановка задачи

Анализ временных рядов нередко предполагает обнаружение моментов, в которые статистические свойства данных претерпевают резкие изменения. Такие точки разладки (change points, CP) соответствуют смене состояния системы или внешних условий и могут существенно влиять на дальнейшее поведение ряда. Задача обнаружения разладок (change point detection – CPD) состоит в том, чтобы по наблюдаемому ряду выявить моменты, когда изменяется закон генерации данных. CPD применяется в задачах мониторинга физических процессов и финансовых систем. В обоих случаях динамику часто моделируют стохастическими дифференциальными уравнениями (SDE) – в физике как описание реального процесса, в финансах как приближение для ценообразования и управления рисками. В данной работе мы используем SDE с синусоидальным потенциалом – модель из статистической физики. Ее математический аппарат (шум-индуцированные переходы между состояниями) переносится и на задачи смены режимов в финансовых рядах.

Есть два режима CPD. В офлайне весь ряд уже перед нами, и мы ищем разладки задним числом – это задача сегментации. В онлайн данные поступают в потоке, и нужно зафиксировать разладку как можно быстрее. Мы работаем в офлайне, хотя некоторые из наших методов (CUSUM) изначально создавались для потока.

1.1 Актуальность проблемы

В реальных данных изменения режима генерации сигнала могут указывать на важные события или структурные сдвиги в системе. Зная, где происходят разладки, можно анализировать каждый режим отдельно, а не усреднять по всему ряду. Прогнозные модели на однородных участках будут точнее. Без детекции точек разладки любая модель, построенная на «склеенных» режимах, рискует дать бессмысленный результат.

Для SDE-траекторий разладка проявляется как перескок процесса между устойчивыми состояниями – скачок уровня, который маскируется шумом и колебаниями вблизи границы между состояниями. Классические тесты предполагают один скачок среднего; нейросети могут выучить более сложный паттерн. Это определяет главный вопрос работы: выяснить, какой метод оптимален при каких параметрах. За десятилетия предложены десятки алгоритмов офлайн-CPD (обзор – в разделе 2), не считая онлайн-методов. У каждого свои предположения и своя ниша, а универсального решения нет. Отсюда мотивация: разобраться, что когда работает. Не абстрактно, а на конкретных данных с контролируемыми параметрами.

1.2 Цель, новизна и задачи

Цель работы – сравнить классические статистические и современные нейросетевые методы обнаружения точек разладки применительно к траекториям стохастических дифференциальных уравнений (SDE) и определить, какие подходы оптимальны в зависимости от параметров динамической системы.

В отличие от большинства бенчмарков CPD, где разладки создаются искусственно (скачок среднего, смена дисперсии), мы работаем с данными из нелинейной SDE вида $dx = \sin(x) dt + \sqrt{D} dW$, где D – коэффициент диффузии (интенсивность шума), dW – приращение винеровского процесса. Здесь разладки возникают сами – процесс проводит время вблизи устойчивых положений и иногда перескакивает между ними

под действием шума. Такая постановка ближе к физическим задачам, где изменение режима – не внешнее вмешательство, а следствие динамики самой системы.

Научная новизна состоит в следующем:

1. Впервые проведено систематическое сравнение широкого набора методов CPD разных классов – от ранговых тестов до Transformer – на траекториях нелинейной SDE, где разладки порождаются динамикой самого процесса, а не вставляются искусственно.
2. Проведен статистический анализ переходов в SDE-процессе при разных уровнях шума и шагах дискретизации. Цель – понять, как параметры генерации влияют на структуру данных и какие методы детекции лучше подходят в каждом режиме.
3. Впервые проведено экспериментальное исследование зависимости оптимального класса методов CPD от уровня шума в SDE-процессе с целью определить границу между режимами.

Задачи:

1. На основе модели из лаборатории научного руководителя (ноутбук с SDE-генератором и схемой Эйлера–Маруямы) реализовать воспроизводимый генератор траекторий с контролируемыми параметрами (D , dt , тип шума) и автоматической разметкой точек разладки.
2. Реализовать и протестировать широкий набор методов CPD разных классов – от ранговых тестов до нейросетей.
3. Провести статистический анализ SDE-процесса: проверить гипотезы о распределении межпереходных расстояний, перелетов и числа разладок при варьировании параметров.
4. Провести полный цикл экспериментов на 4 сценариях (разные D и типы шума) с 50 тестовыми рядами на метод и строгим разделением обучение/калибровка/тест.
5. Сформулировать практические рекомендации: какой класс методов применять в зависимости от характеристик данных.

2 Обзор литературы и мотивация

2.1 Общие сведения о CPD

Первый формальный метод CPD – CUSUM – предложен Page [1] для контроля качества на производстве. С тех пор область разрослась: Truong et al. [17] насчитали более 140 алгоритмов офлайн-CPD, разложив каждый на три блока: критерий однородности сегмента, процедура поиска разбиения и регуляризация числа разрывов. Их библиотека `ruptures` на Python стала стандартным инструментом. Aminikhanghahi и Cook [14] дают более широкий обзор, включая онлайн-методы, и подчеркивают: CPD ищет момент смены закона генерации, а не просто выброс – это другая задача, нежели *anomaly detection*. Систематических сравнений на данных с нелинейной динамикой при этом нет.

2.2 Категории методов

Методы CPD принято делить на четыре класса, хотя границы условны. Для нашей задачи ключевые два.

Первый – **тесты на однородность**: проверяют, различаются ли параметры ряда до и после предполагаемой точки. Сюда входят ранговые критерии (Петтитт, Манн-Уитни), кумулятивные схемы (CUSUM) и тесты из климатологии (SNHT – Standard Normal Homogeneity Test, Буишанд). Большинство ищут одну точку за прогон – на рядах с десятками разладок это ограничение.

Второй – **методы скользящих окон**, сравнивающие поведение ряда «до» и «после» текущего момента по мерам различия между распределениями или отношению плотностей. Методы оценки отношения плотностей [12] оценивают $p_{\text{after}}/p_{\text{before}}$ напрямую. Все наши ML-методы тоже работают в этой парадигме: CatBoost сравнивает статистики половин окна, нейросети обучаются различать «окно с разладкой» от «окна без». Главная сложность на практике – выбор ширины окна.

Отдельно стоят **алгоритмы оптимальной сегментации** (PELT [11], MDL) – они разбивают ряд на однородные куски, штрафую за лишние разладки. **Байесовские подходы** (BOCPD [8]) дают вероятность разладки в каждой точке, но взамен требуют задать модель данных и априорное распределение длительности сегмента (функцию, задающую вероятность разладки в каждый момент при условии, что до сих пор её не было – hazard function). Ещё один подход – автоэнкодеры (ALACPD [19]): они учатся восстанавливать нормальный сигнал, и разладку фиксируют по росту ошибки восстановления.

2.3 Сравнительные исследования

Прямых сравнений CPD-методов немного. Те что есть [17, 12, 18] проведены на кусочно-стационарных данных – когда ряд искусственно переключается между режимами – и приходят к одному выводу: лучший метод зависит от данных. При этом ни одно сравнение не проводилось на данных, где разладки возникают из динамики самой системы, а не вставляются вручную. Наша работа закрывает этот пробел.

2.4 Что следует из обзора

Обзор определяет три ключевых параметра эксперимента.

Методов много, и они очень разные. Необходимо охватить методы разных классов – от ранговых тестов до нейросетей, – чтобы не заузить сравнение. Ряд подходов (PELT, BOCPD, RuLSIF, Kernel CPD) сознательно оставлен за рамками: PELT превращает сравнение в подбор функции стоимости и штрафа, BOCPD требует задания hazard-функции, а RuLSIF и Kernel CPD концептуально близки к нашему CatBoost (сравнение окон «до» и «после»), но без обучения на данных. Метрик тоже хватает: одни работы смотрят на задержку обнаружения, другие – на precision/recall, третьи – на ROC-AUC. Мы используем шесть метрик параллельно, потому что ни одна из них в отдельности не дает полной картины.

Тестовые данные в литературе почти всегда кусочно-стационарные: ряд $\mathcal{N}(0, 1)$, потом $\mathcal{N}(3, 1)$, и вот вам changepoint. Это удобно, но далеко от реальных задач, где разладки порождаются динамикой самой системы, а не внешним переключением. Мы используем траектории SDE, в которых переходы между устойчивыми состо-

яниями возникают естественно – и это, насколько нам известно, первое сравнение CPD-методов на таких данных.

Одна из центральных гипотез, вынесенных из обзора: нейросети должны побеждать на сложных паттернах, но проигрывать на простых. Эту гипотезу мы проверяем экспериментально в разделе 7.

3 Обзор методов CPD

1. **CUSUM (Cumulative Sum)** – классический онлайн-метод Page [1], накапливающий отклонения от ожидаемого среднего и срабатывающий при превышении порога. Единственный классический метод в нашем наборе, способный находить множественные CP: CUSUM последовательно обнаруживает десятки разладок, тогда как single-CP тесты (SNHT, Chow) находят одну. Расплата за высокий recall – ложные срабатывания: CUSUM реагирует на любой сдвиг среднего, в том числе вызванный шумом.
2. **CatBoost (градиентный бустинг на признаках окна)** – из каждого окна длиной W извлекаются 14 статистик (полный список – в `catboost_cpd.py`), на которых обучается `CatBoostClassifier`. Ключевые четыре:

$\bar{x}_L - \bar{x}_R$	разность средних левой и правой половин окна
$\sigma_L - \sigma_R$	разность стандартных отклонений половин
slope	наклон линейного тренда внутри окна
r_1	автокорреляция с лагом 1

Признаки $\bar{x}_L - \bar{x}_R$ и $\sigma_L - \sigma_R$ – самые информативные: если в центре окна есть разладка, левая и правая половины будут различаться по уровню или разбросу. CatBoost быстро учится (минуты на CPU) и позволяет через оценку важности признаков (feature importance) увидеть, какие статистики «ловят» разладку. Главное ограничение – фиксированный набор из 14 признаков. Если разладка проявляется через статистику, не вошедшую в набор, метод ее пропустит. Нейросети этого ограничения лишены. CatBoost не имеет v2-версии: при обучении на v2-данных ($D \in [0.2, 1.0]$, $W = 100$) результат улучшился незначительно, и мы оставили единственную конфигурацию ($W = 50$, 500 деревьев).

3. **Рекуррентные нейросетевые подходы к CPD: LSTM и GRU** – класс методов, в которых задача детекции разладки формулируется как бинарная классификация скользящего окна рекуррентной сетью. В нашей реализации оба метода используют единую схему: окно фиксированной длины нормируется и подается на вход рекуррентной сети, которая по последнему скрытому состоянию предсказывает вероятность разладки в центре окна; модели обучаются на синтетических данных с разными типами шума и параметрами, после чего применяются к тестовому ряду поточечно, а конечное множество точек разладки извлекается через пороговую бинаризацию и подавление немаксимумов (NMS).

LSTM (Long Short-Term Memory) оснащен механизмом памяти ячейки (cell state) и тремя вентилями (входным, забывания и выходным), что позволяет явно управлять долгосрочной памятью и потенциально полезно для улавливания отложенных или постепенных разладок. Модель содержит больше параметров и требует существенно большего времени обучения.

GRU (Gated Recurrent Unit) упрощает LSTM до двух вентилях – сброса и обновления – без отдельного cell state. GRU имеет меньше параметров и обучается быстрее при сопоставимом качестве на контекстах средней длины. GRU включен как контрольное сравнение: если он покажет результат, близкий к LSTM, значит, дело в самой рекуррентности, а не в механизме cell state. У обоих методов общие слабости: им нужно много обучающих данных, и они могут переобучиться на конкретный тип шума. Кроме того, в отличие от классических тестов, у нейросетей нет формальных гарантий на долю ложных срабатываний – порог подбирается эмпирически.

4. **Transformer-подходы к CPD** – идея та же, что у LSTM: скользящее окно, нейросеть, score. Разница в архитектуре: механизм self-attention позволяет Transformer сопоставлять далёкие участки ряда напрямую, без последовательной обработки. Разладку можно ловить двумя способами – по росту ошибки прогноза (модель не может предсказать следующий шаг, значит что-то изменилось) или по резкой смене внутренних представлений сети. Главный минус – обучение ресурсоёмко (около часа на GPU (Kaggle T4) против минут у CatBoost), а на рядах с редкими разладками модель может быть неустойчива (раздел 7).

Помимо CUSUM (описанного выше) и ML-методов, в набор вошли 11 классических статистических тестов. Краткое описание каждого приведено в табл. 1.

Все single-CP тесты (Pettitt, SNHT, Buishand, Chow, GP, Nyblom) ищут ровно одну разладку за прогон. MDL и SWAB могут находить несколько. Итого реализовано 18 методов (12 классических + 6 ML). Из классических в основных таблицах результатов обсуждаются SNHT, Chow, Pettitt, CUSUM и Buishand – они либо показали содержательные результаты, либо интересны как baseline. GP ($F1 < 0.02$ при $D \geq 1.0$), SWAB ($F1 < 0.12$), MDL ($F1 < 0.30$) и Nyblom ($F1 < 0.05$) протестированы, но при рабочих D практически не работают: single-CP архитектура и отсутствие адаптации к множественным разладкам делают их неприменимыми.

Все методы приведены к единому интерфейсу: на вход подается временной ряд и параметры, на выходе – список предсказанных позиций разладок, массив score (если метод его выдает) и словарь метаданных. Это позволило автоматизировать оценку: один скрипт (`eval_multi.py`) прогоняет все методы на заданном числе тестовых рядов и собирает метрики в единую таблицу.

3.1 Детали архитектуры и обучения ML-моделей

Все нейросетевые модели в нашей реализации работают по единой схеме «скользящее окно $\rightarrow$ нейросеть $\rightarrow$ score $\rightarrow$ порог + NMS».

Скользящее окно. Входной временной ряд $x_1, \dots, x_T$ обрабатывается окном фиксированной длины W . Для каждого t формируется подпоследовательность $\mathbf{w}_t = (x_{t-W/2}, \dots, x_{t+W/2})$, которая стандартизуется: $\hat{\mathbf{w}}_t = (\mathbf{w}_t - \bar{\mathbf{w}}_t) / \sigma(\mathbf{w}_t)$. Стандартизация нужна, чтобы модель реагировала на *форму* сигнала в окне, а не на его абсолютный уровень – иначе при дрефте траектории модель будет «путать» сдвиг уровня с разладкой.

Нейросетевой классификатор. Стандартизованное окно подаётся на вход сети. LSTM и GRU читают его шаг за шагом, накапливая информацию в скрытом состоянии. Когда окно прочитано, последнее скрытое состояние содержит «сводку»

Таблица 1: Классические статистические методы CPD в нашем наборе. CP: 1 = находит одну разладку, n = несколько. Score: + = выдает непрерывный score для ROC-AUC.

Метод	CP	Score	Принцип работы
Pettitt	1	–	Ранговый тест [2]: статистика Манна – Уитни для всех точек разделения, выбирается максимум.
SNHT	1	–	Сравнение средних до и после каждой точки, нормированное на общую дисперсию [4].
Buishand	1	+	Тест однородности на кумулятивных отклонениях от среднего. Реализовано 4 варианта (Q, Range, LR, U); все дали одинаковый F1, далее приводим LR.
Chow	1	+	F-тест структурного разрыва: сравнивает регрессии до и после точки.
MDL	n	–	Minimum Description Length [3]: разрыв добавляется, если сокращает длину описания.
SWAB	n	–	Sliding Window and Bottom-Up [6]: скользящее окно + слияние сегментов.
GP	1	–	Gaussian Process [10]: детекция через маргинальное правдоподобие.
Nyblom	1	+	Проверяет, остаются ли параметры ряда (среднее, дисперсия) постоянными. В отличие от SNHT и Chow, рассчитан на плавные сдвиги, а не резкие скачки.

всего увиденного – её пропускают через линейный слой и сигмоиду, чтобы получить одно число. Transformer устроен иначе: он видит всё окно сразу. Каждый шаг ряда становится отдельным токеном с позиционным кодированием (чтобы сеть знала порядок), и блоки self-attention сами решают, какие участки окна сопоставить друг с другом. Результат агрегируется и тоже сводится к одному числу. В обоих случаях на выходе – вероятность $p_t \in [0, 1]$, что в центре окна находится разладка.

Постобработка. Массив вероятностей $\{p_t\}$ сначала бинаризуется порогом θ (различным для каждого метода, калиброванным на отдельном наборе данных), затем проходит через подавление немаксимумов (NMS): если два срабатывания находятся ближе заданного расстояния друг к другу (50 шагов для $v1$, 10 для $v2$), из пары остается то, у которого p_t выше. Это предотвращает «кластеризацию» ложных срабатываний вокруг одной разладки.

Обучение. Модели обучались на синтетических SDE-рядах с функцией потерь бинарной кросс-энтропии (BCE) – она штрафует модель тем сильнее, чем увереннее та ошиблась: если истинная метка «разладка», а модель выдала вероятность близкую к нулю, штраф будет большим. Для каждого обучающего ряда из него нарезались окна с перекрытием, метка $y_t = 1$, если центр окна находится в пределах 5 шагов от истинной разладки (обучающий допуск – не путать с $\delta = 50$ при оценке качества в разделе 5), иначе $y_t = 0$. Обучающий допуск узкий: модель должна учиться выдавать высокий score именно вблизи разладки, а не на всем ряде. Дисбаланс классов (раз-

ладок на порядок меньше, чем «обычных» шагов) компенсировался весами классов при подсчёте лосса.

Конкретные архитектуры и гиперпараметры приведены в табл. 2.

Таблица 2: Архитектуры и гиперпараметры ML-моделей. H – размер скрытого слоя, L – число слоёв, W – ширина окна, LR – learning rate (скорость обучения: насколько сильно модель корректирует веса на каждом шаге; слишком большой – обучение нестабильно, слишком маленький – застревает).

Модель	H	L	W	Epochs*	LR	Порог θ
LSTM	256	2	50	300	0.001	0.89
LSTM_v2	256	2	100	300	0.001	0.30
GRU	128	2	50	100	0.001	0.99
GRU_v2	128	2	100	100	0.001	0.30
CatBoost	–	–	50	500	0.05	0.57
Transformer_v2	256	6	100	100	0.001	0.49

* Для CatBoost «Epochs» означает число раундов бустинга (iterations), а не эпохи в смысле нейросетей.

Пороги θ заслуживают внимания: у моделей v1 они близки к единице (0.89 – 0.99), что означает «срабатывать только при полной уверенности». Это следствие обучения на слишком «легких» данных: модель всегда видела много переходов и научилась предсказывать их уверенно, а на тестовых данных с редкими переходами порог приходилось задирать, чтобы хоть как-то снизить шквал ложных срабатываний. У v2-моделей пороги значительно ниже (0.30 – 0.49) – модели обучены на реалистичных D и могут позволить себе быть чувствительнее, не теряя precision.

Во-вторых, ширина окна: v2-модели используют $W = 100$ вместо 50. Удвоение окна дает модели больше контекста для принятия решения, что особенно важно при SDE-данных, где переходы между устойчивыми состояниями могут сопровождаться осцилляциями длиной в десятки шагов.

В-третьих, Transformer_v2 – самая «тяжелая» модель: 6 слоев с 8 головами внимания и $d_{\text{model}} = 256$ дают около 3.2 миллиона обучаемых параметров. Для сравнения, LSTM_v2 содержит порядка 800 тысяч, а GRU_v2 – около 150 тысяч параметров. Несмотря на разницу в размере, время инференса сопоставимо (несколько секунд на ряд длиной 2000), поскольку основная стоимость – последовательный проход окнами, а не собственно вычисление сети.

3.2 Сравнение версий: v1 и v2

Ключевое отличие v2 от v1 – состав обучающей выборки. Модели v1 обучались на данных с $D \in [0.05, 5.0]$ (случайный из диапазона). Проблема: при D ближе к верхней границе на 5000 шагов приходится в среднем 750 переходов – доля разладок около 15%. А при тестировании на $D = 1.0$ доля разладок составляет $\sim 2\%$. Разрыв в 7 – 8 раз: модель привыкла видеть разладки на каждом шагу и на тестовых данных «галлюцинирует» лишние. Модели v2 обучались на выборке с $D \sim \text{Uniform}(0.2, 1.0)$ и $dt \in \{0.5, 1.0, 2.0\}$, что приближает обучающие условия к тестовым. Кроме того, каждый обучающий ряд генерировался со случайно выбранным типом шума (белый, розовый, броуновский, фиолетовый, голубой) – чтобы модель не переобучалась на один спектр помех. Transformer обучался только в конфигурации v2: к моменту его

реализации проблема разрыва статистик была уже выявлена на LSTM и GRU, и повторять ошибку не имело смысла. Количественно при $D = 1.0$: LSTM v1 F1 = 0.402 $\rightarrow$ v2 F1 = 0.675; GRU v1 F1 = 0.359 $\rightarrow$ v2 F1 = 0.650 – рост в 1.7–1.8 раза от одного изменения обучающей выборки.

4 Синтетические данные на основе SDE

Все эксперименты проведены на синтетических временных рядах, полученных численным интегрированием SDE. Исходная модель генерации (SDE с sin-потенциалом, схема Эйлера–Маруямы, генераторы цветного шума) взята из ноутбука лаборатории научного руководителя. На ее основе мы реализовали воспроизводимый модуль `data_utils.py` с параметризацией по D , dt , типу шума и seed, автоматической разметкой CP и единым API для обучения и тестирования. Синтетика дает полный контроль над разметкой и параметрами генерации; ограничения этого выбора обсуждаются в разделе 8.

4.1 Модель генерации

В основе генерации лежит одномерное SDE с синусоидальным дрейфом:

$$dx = \sin(x) dt + \sqrt{D} dW, \quad (1)$$

где D – коэффициент диффузии, dW – приращение винеровского процесса. Численное интегрирование проводится по схеме Эйлера – Маруямы – стандартному методу дискретизации SDE, аналогу метода Эйлера для обыкновенных дифференциальных уравнений, но с добавлением шумового слагаемого:

$$x_{t+1} = x_t + \sin(x_t) \Delta t + \sqrt{D} \xi_t, \quad (2)$$

где ξ_t – нормализованный шум ($\mathbb{E}[\xi] = 0$, $\text{Var}[\xi] = 1$). В стандартной схеме Эйлера–Маруямы шумовой член имеет вид $\sqrt{D \cdot \Delta t} \xi_t$; в нашей реализации множитель $\sqrt{\Delta t}$ поглощен нормализацией шума, поэтому при $\Delta t = 1$ формулы совпадают. Для белого шума ξ_t независимы, для цветного (розового, броуновского) – коррелированы, но нормализованы по дисперсии. Начальное условие: $x_0 = \pi/10^6 \approx 0$.

Функция $\sin(x)$ задает «ландшафт», по которому движется процесс. Дрейфт $f(x) = \sin(x)$ соответствует потенциалу $V(x) = \cos(x)$ (через соотношение $f(x) = -V'(x)$). Впадины (минимумы потенциала) расположены в точках $x = (2k + 1)\pi$ (нечетные кратные π : $\pi, 3\pi, 5\pi, \dots$) – это устойчивые положения, где процесс «хочет» оставаться. Вершины холмов в $x = 2k\pi$ (четные: $0, 2\pi, 4\pi, \dots$) – неустойчивые: процесс там не задерживается. Большую часть времени траектория колеблется вблизи дна одной впадины, но шум $\sqrt{D} dW$ иногда «выталкивает» её через вершину в соседнюю – это и есть разладка. Точка разладки определяется как момент, когда $\lfloor x_t/\pi \rfloor$ изменяется – то есть траектория пересекает границу, кратную π . Это дает естественную разметку: переход из одной впадины потенциала в другую соответствует качественной смене режима процесса.

4.2 Параметры и их влияние

Коэффициент диффузии D – ключевой параметр, определяющий сложность задачи CPD. При $D = 0.5$ на тестовом ряде длиной 2000 шагов в среднем ~ 2 перехода

– задача тривиальна, и классические тесты справляются. С ростом D до 1.0 число переходов увеличивается до ~ 200 , паттерны усложняются, но процесс все еще проводит заметное время в каждой впадине. При $D = 2.0$ процесс скачет почти непрерывно (~ 1000 переходов), и отличить настоящий переход от осцилляции у границы π – уже отдельная проблема.

Шаг дискретизации dt определяет масштаб времени между соседними наблюдениями. По умолчанию мы используем $dt = 1.0$ – единичный шаг, при котором одно наблюдение соответствует одному условному моменту времени. Это естественный выбор для безразмерной модели: мы не привязаны к конкретным физическим единицам и можем управлять сложностью задачи через коэффициент диффузии D , не трогая временной масштаб. Влияние dt на статистику переходов мы тем не менее исследовали отдельно (раздел 6, сетка из 15 значений от 0.2 до 3.0): при малом dt траектория ближе к непрерывному пределу и переходы «размазываются», при большом – процесс прыгает на значительные расстояния за один шаг. Для сравнения методов детекции dt зафиксирован на 1.0, поскольку одного параметра D достаточно для покрытия всего спектра сложности.

Шум в SDE всегда строится из нормального: на каждом шаге генерируется независимое $\xi_t \sim \mathcal{N}(0, 1)$. Белый шум – это сами ξ_t без изменений. Цветной шум получается фильтрацией: последовательность ξ_t преобразуется в частотной области так, чтобы усилить или подавить определённые частоты (здесь f – частота), а затем нормализуется по дисперсии (чтобы амплитуда оставалась сопоставимой). Розовый ($\sim 1/\sqrt{f}$) и броуновский ($\sim 1/f$) усиливают низкие частоты – траектория становится более плавной, переходы между впадинами выглядят иначе. Фиолетовый ($\sim f$) и голубой ($\sim \sqrt{f}$), наоборот, усиливают высокие – траектория более «дёрганая». Пять типов шума нужны, чтобы проверить, насколько методы детекции чувствительны к характеру шума, а не только к его амплитуде.

5 Протокол экспериментов

Тестовые ряды генерируются длиной 2000 шагов. При $D = 1.0$ это соответствует в среднем ~ 40 разладок на ряд. Рис. 1 иллюстрирует типичную SDE-траекторию при $D = 1.0$ и результаты нескольких методов детекции.

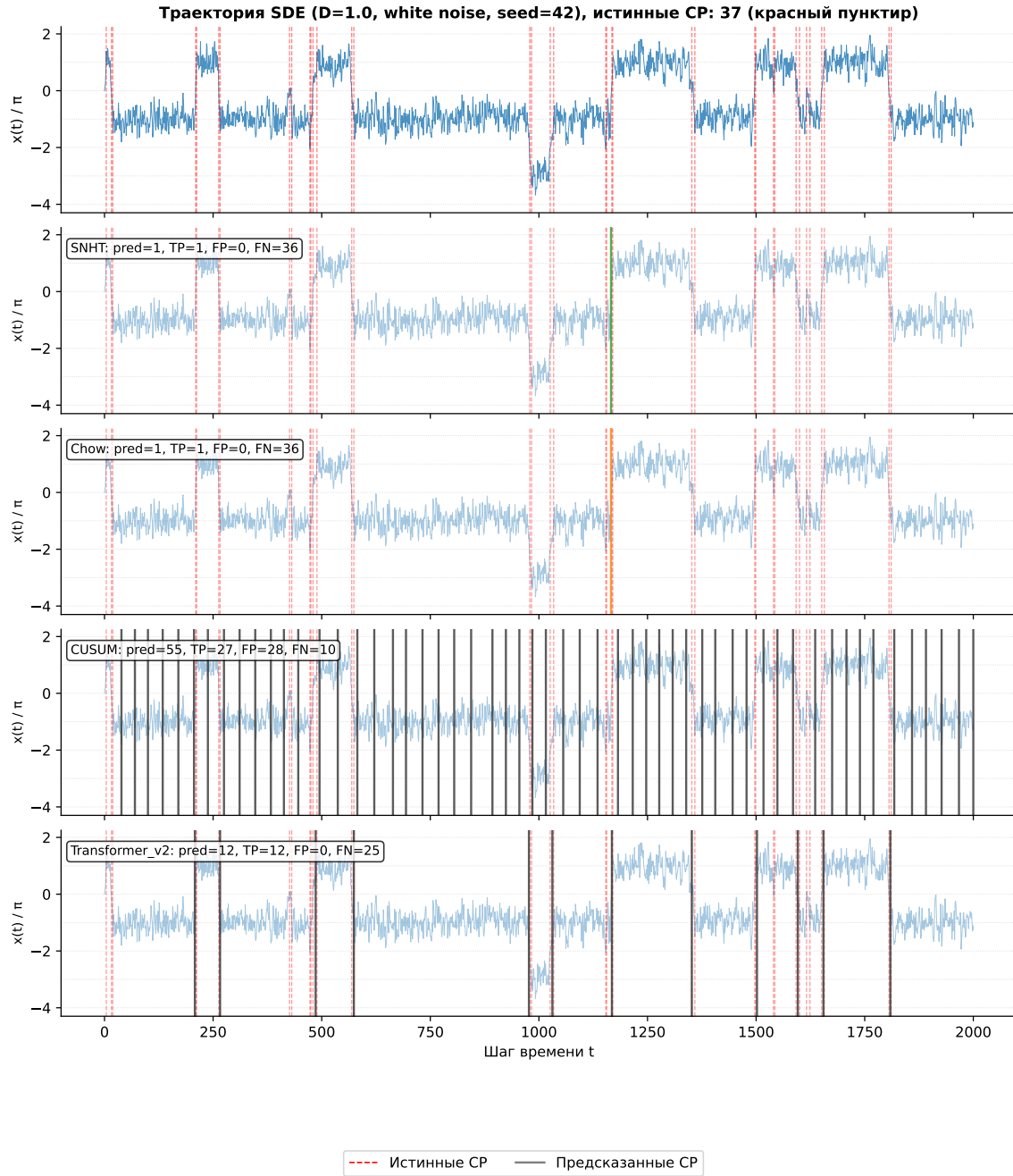


Рис. 1: SDE-траектория ($D = 1.0$, белый шум, 2000 шагов, 37 истинных CP). Красный пунктир – истинные точки разладки. Цветные сплошные – предсказания методов. SNHT и Chow находят по одной точке; CUSUM – 55 (много ложных); Transformer_v2 – 12 (консервативно, но точно).

5.1 Метрики качества

Алгоритм CPD почти никогда не попадает ровно в истинную позицию разладки, поэтому вводится допуск δ : предсказание засчитывается как TP, если расстояние до ближайшей истинной точки не превышает δ шагов (конкретное значение и процедура сопоставления описаны ниже). На этой основе считаются стандартные метрики:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3)$$

F1 – основная метрика; Ассигасу не используется (при $TN \gg TP$ алгоритм, всегда отвечающий «нет», получит 99%+). Для методов с непрерывным score дополнительно считаются **ROC-AUC** (качество ранжирования без фиксации порога) и **PR-AUC** (более честная метрика при дисбалансе классов).

Классификационные метрики и метрики локализации мы держим параллельно: на практике точность определения точки и факт детекции – разные задачи. Для локализации используется MAE:

$$MAE = \frac{1}{M} \sum_{i=1}^M |\hat{\tau}_i - \tau_i|, \quad (4)$$

где τ_i – истинное время i -й разладки, $\hat{\tau}_i$ – предсказанное. MAE показывает среднее отклонение в шагах.

5.2 Разделение данных

Корректное сравнение методов требует строгого разделения данных на три непересекающихся набора:

- **Обучающий набор** (seeds 0 – 99): используется только для обучения ML-моделей. Параметры генерации варьируются ($D \in [0.2, 1.0]$, $dt \in \{0.5, 1.0, 2.0\}$, все 5 типов шума), чтобы модели не переобучались на один режим.
- **Калибровочный набор** (seeds 5000 – 5019, 20 рядов): используется для подбора порогов. Для ML-моделей максимизируется F1 по сетке из 95 значений $threshold \in [0.05, 0.99]$. Для классических тестов используются стандартные уровни значимости.
- **Тестовый набор** (seeds 1000 – 1049, 50 рядов по 2000 шагов): используется только для финального сравнения. Ни один параметр не подбирался на этих данных.

Калибровка порогов – критический этап: различия в процедуре калибровки между v1 и v2 описаны в разделе 3.2, влияние на результаты – в разделе 7.

5.3 Сопоставление и сценарии

Сопоставление предсказаний с истиной. Предсказанная точка считается верной (TP), если находится в пределах $\delta = 50$ шагов от ближайшей истинной разладки: при $D = 1.0$ среднее расстояние между соседними разладками составляет ~ 50 шагов, так что $\delta = 50$ означает «предсказание засчитывается, если оно ближе к правильной разладке, чем к соседней». Сопоставление – жадное: пары (true, pred) сортируются по расстоянию, каждая точка используется не более одного раза. Непарные предсказания – FP, непарные истинные – FN. Жадное сопоставление может давать субоптимальные пары по сравнению с венгерским алгоритмом; мы не проверяли разницу формально, но при $\delta = 50$ и типичных расстояниях между CP она пренебрежимо мала.

Допуск $\delta = 50$ не следует путать с NMS-расстоянием (50 для v1-моделей, 10 для v2): δ используется при сопоставлении предсказаний с разметкой, NMS – при постобработке score-функции. NMS для v2 уменьшено, поскольку их score-функции менее

кластеризованы; δ при этом не менялся, так как он определяется не свойствами метода, а типичным расстоянием между истинными СР. Единый $\delta = 50$ для всех сценариев – осознанный компромисс. При $D = 0.5$ среднее расстояние между СР ~ 1000 шагов, и $\delta = 50$ составляет лишь 5% от него – допуск очень щедрый. При $D = 1.0$ расстояние ~ 50 шагов, и δ совпадает с типичным межпереходным интервалом: предсказание может «притянуться» к соседней разладке. Однако жадное сопоставление решает эту проблему – ближайшие пары связываются первыми, и каждая точка используется однократно. При $D = 1.5$ расстояние ~ 17 шагов, и $\delta = 50$ покрывает 2 – 3 соседних разладки, но опять же однократное использование не позволяет одному предсказанию «забрать» несколько истинных точек. Уменьшение δ ниже 50 наказывало бы методы с неточной локализацией ($\text{MAE} > 5$), но не меняло бы ранжирование.

Сценарии. Каждый метод протестирован в четырех условиях: $D = 0.5$ (white), $D = 1.0$ (white), $D = 1.0$ (pink), $D = 1.5$ (white). Это покрывает спектр от тривиальной задачи (~ 2 СР на ряд при $D = 0.5$) до сложной (~ 115 СР при $D = 1.5$, белый шум; ~ 191 СР при $D = 1.0$, розовый шум) и позволяет оценить устойчивость методов к типу шума.

Статистическая надежность. 50 независимых реализаций на сценарий дают возможность оценить не только среднее, но и разброс метрик.

5.4 Устройство пайплайна

Весь процесс автоматизирован в скрипте `eval_multi.py`:

1. Генерация тестового ряда с заданными параметрами (D , dt , noise, seed).
2. Последовательный запуск всех 18 методов через единый интерфейс.
3. Сопоставление предсказаний с разметкой, вычисление метрик.
4. Повторение для 50 сидов, агрегация в `summary.csv`.

Для ML-моделей адаптер проверяет наличие файла весов и при его наличии пропускает обучение – полный eval занимает минуты, а не часы.

5.5 Исходные гипотезы

Перед запуском экспериментов сформулированы семь ожиданий, проверяемых в разделе 7:

1. Классические тесты лидируют при малом шуме (редкие, выраженные разладки).
2. ML-модели лидируют при сильном шуме (частые, зашумленные переходы).
3. Коррелированный шум сильнее бьет по классике, чем по ML.
4. Существует критический уровень шума, разделяющий режимы.
5. CUSUM дает высокий recall при низком precision.
6. Модели v2 значительно лучше v1.
7. Нейросетевые методы превосходят классику при всех уровнях шума.

6 Статистический анализ SDE-процесса

Протокол зафиксирован, но перед запуском детекции мы провели отдельное исследование: как именно устроены переходы в SDE-процессе при разных D и dt . Это нужно, чтобы понять, в каком режиме задача тривиальна, а в каком – содержательна, и обосновать выбор тестовых сценариев.

Мы задали себе три вопроса:

- Как выглядят «перелёты» через границу между впадинами? (Н1)
- Сколько времени процесс проводит в одной впадине до перескока? (Н2)
- Случайно ли расположены разладки, или между ними есть зависимость? (Н3)

Анализ проведен на сетке из 20 значений D и 15 значений dt (всего 300 комбинаций, по 30 рядов на каждую).

6.1 Н1: Перелеты через границу – half-normal

Когда траектория пересекает границу $k\pi$, она «перелетает» на некоторое расстояние $|x_t - k\pi|$. Если шум мал, этот перелет определяется одним гауссовским шагом, и его абсолютная величина должна следовать полунормальному распределению (half-normal) – это прямое следствие нормальности шумового приращения: $|\xi| \sim \text{half-normal}$ при $\xi \sim \mathcal{N}(0, 1)$.

Чтобы проверить, действительно ли перелёты следуют полунормальному распределению, нужен формальный критерий – нельзя просто посмотреть на гистограмму и сказать «похоже». Мы используем критерий согласия хи-квадрат (χ^2). Идея простая: разбиваем наблюдаемые перелёты на интервалы (бины), считаем сколько значений попало в каждый, и сравниваем с тем, сколько *должно было* попасть, если распределение действительно полунормальное. Мера расхождения – статистика $\chi^2 = \sum (O_i - E_i)^2 / E_i$, где O_i – наблюдаемое число в i -м бине, E_i – ожидаемое. Чем больше χ^2 , тем хуже совпадение.

По значению χ^2 вычисляется p -value – вероятность увидеть такое же или большее расхождение чисто по случайности, если распределение на самом деле полунормальное. Если $p > 0.05$ – расхождение легко объяснить случайностью, и у нас нет оснований отвергать гипотезу. Если $p < 0.05$ – расхождение слишком велико, полунормальное распределение не подходит.

Формально тест не отвергает гипотезу при малом шуме ($D \leq 0.6$, рис. 2), но этот результат обманчив: переходов при таком D мало (десятки – сотни), и у теста просто недостаточно данных, чтобы отвергнуть что бы то ни было. Гистограмма при $D = 0.2$ (рис. 3, левая панель) выглядит рваной и плохо совпадает с теоретической кривой, хотя формально $p = 0.91$.

При более сильном шуме ($D = 1.0$, рис. 3, третья панель) ситуация обратная: визуально гистограмма ложится на кривую гораздо лучше, но тест отвергает гипотезу ($p \approx 0$), потому что данных тысячи и даже крошечное отклонение становится «статистически значимым».

Содержательный вывод: полунормальное распределение – разумная аппроксимация перелётов при любом D . Для практики важнее не формальный p -value, а сам факт: форма распределения перелётов меняется с D предсказуемым образом, что помогает выбрать параметры генерации.

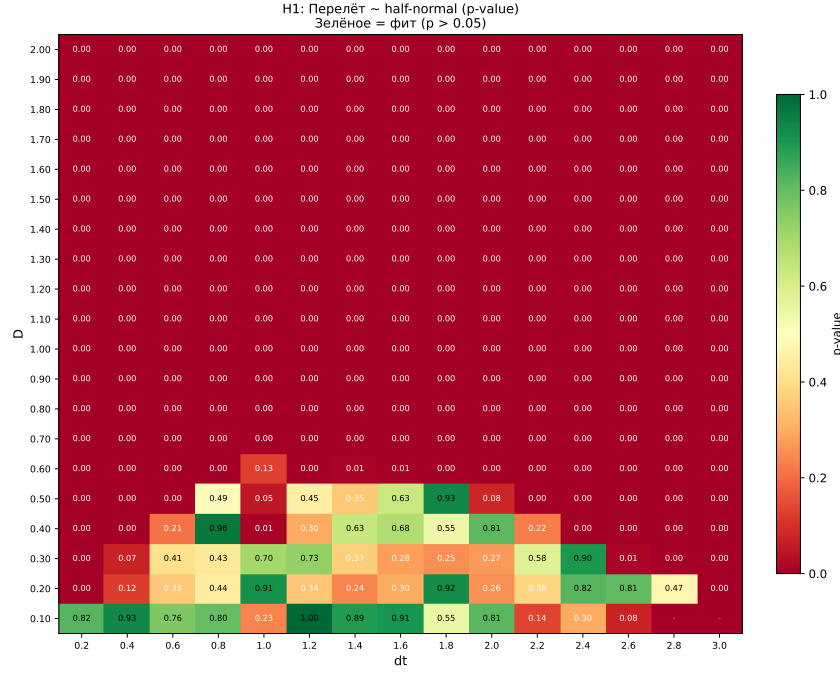


Рис. 2: H1: p -value хи-квадрат теста на полунормальность перелетов. Зеленые ячейки ($p > 0.05$) – гипотеза формально не отвергается, но при малых D это следствие малого объема данных ($n = 30 - 200$), а не хорошего совпадения. При $D \geq 1.0$ тест отвергает из-за больших выборок ($n > 5000$), хотя визуально совпадение лучше (см. рис. 3).

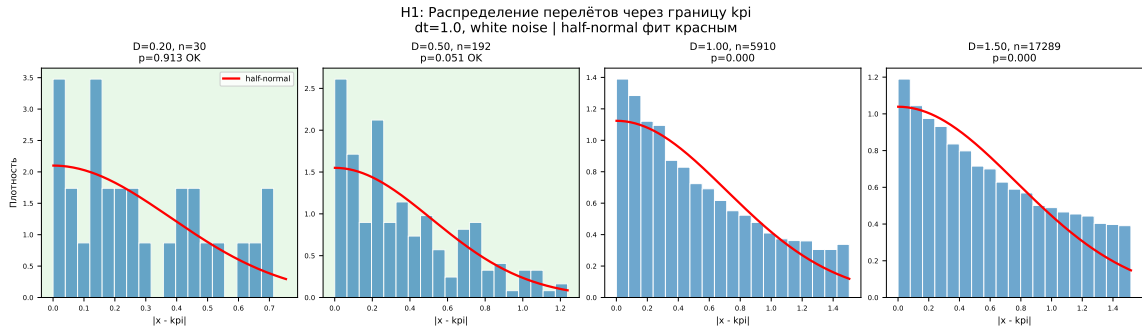


Рис. 3: Гистограммы перелетов через границу $k\pi$ при разных D ($dt = 1.0$). Красная кривая – подгонка полунормальным распределением. При $D = 0.2$ совпадение отличное; при $D = 1.0$ и выше распределение отклоняется.

6.2 H2: Форма распределения временных расстояний между СР

Сколько времени процесс проводит в одной впадине до перескока в следующую? Чтобы ответить, мы измерили расстояния (в шагах) между последовательными точками разладки и подгоняли к ним гамма-распределение.

Гамма-распределение – гибкое семейство, форма которого управляется одним числом: параметром k (shape). Этот параметр – удобный индикатор режима процесса:

- $k < 1$: распределение скошено вправо – процесс чаще «перескакивает» быстро, но иногда застревает надолго.
- $k = 1$: экспоненциальное распределение – переходы происходят «случайно и независимо», процесс не «помнит», как давно он в этой впадине.
- $k > 1$: колоколообразное – время пребывания сконцентрировано вокруг типичного значения.

На рис. 4 видно, как k растёт с увеличением шума (значение k указано в заголовке каждой панели):

- $D \approx 0.5$, $k \approx 0.27$: распределение сильно скошено вправо – процесс обычно сидит в одной впадине долго, но изредка перескакивает быстро.
- $D \approx 1.0$, $k \approx 0.5$: скошенность уменьшается, гамма-распределение хорошо описывает основное тело данных (рис. 4, вторая панель).
- $D \approx 2.0$, $k \approx 1.0$: форма приближается к экспоненциальной – переходы происходят хаотично, процесс не «застревает» надолго.

Визуально гамма-распределение описывает данные тем лучше, чем больше D : при $D \geq 1.0$ подогнанная кривая практически совпадает с гистограммой. При малом D совпадение хуже – данные устроены как смесь двух режимов: короткие осцилляции у границы π (процесс перескочил и тут же вернулся) и длинные удержания в глубине впадины. Такую смесь одно гамма-распределение не описывает, но параметр k всё равно полезен как индикатор режима: его рост от 0.27 до 1.0 отражает переход от редких крупных скачков к стационарному потоку мелких переходов.

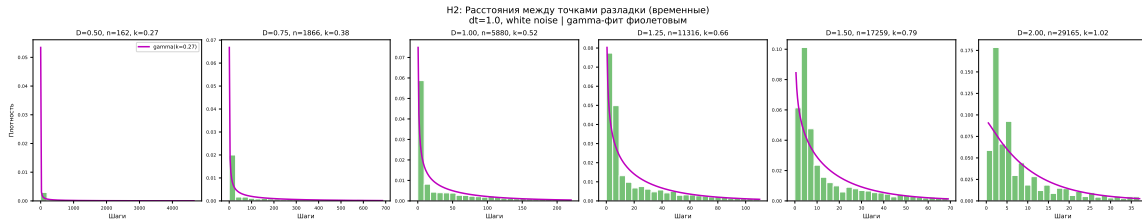


Рис. 4: Гистограммы временных расстояний между СР при разных D . Фиолетовая кривая – подгонка гамма-распределением. При малом D распределение сильно скошено вправо, при большом – приближается к экспоненциальному.

6.3 НЗ: Пуассоновость числа СР

Случайно ли расположены разладки вдоль ряда, или между ними есть зависимость? Если число переходов на каждом ряде подчиняется распределению Пуассона, это означает, что разладки возникают «независимо друг от друга» – как редкие события, не влияющие одно на другое.

Для проверки мы использовали индекс дисперсии – отношение дисперсии числа разладок к их среднему числу: $DI = \text{Var}(N_{\text{CP}}) / \text{Mean}(N_{\text{CP}})$. У пуассоновского распределения дисперсия равна среднему, поэтому $DI \approx 1$ – признак пуассоновского поведения. Отклонения от единицы содержательны: $DI > 1$ (передисперсия) означает, что разладки кластеризуются – на одних рядах их непропорционально много, на

других почти нет; $DI < 1$ (неодисперсия) – что разладки распределены подозрительно равномерно, как будто между ними есть «отталкивание». Для ML-моделей важно именно $DI \approx 1$: это гарантирует, что статистики обучающей и тестовой выборки будут сопоставимы.

Результат (рис. 5): при достаточном шуме ($D \geq 1.2$) индекс дисперсии стабильно близок к единице ($DI \in [0.8, 1.2]$). Это хорошая новость для ML-моделей: в рабочем диапазоне ($D = 1.0 - 1.5$) разладки распределены стационарно, и модель может рассчитывать, что статистики обучения и теста совпадут. При малом D переходов на ряд слишком мало для осмысленной проверки.

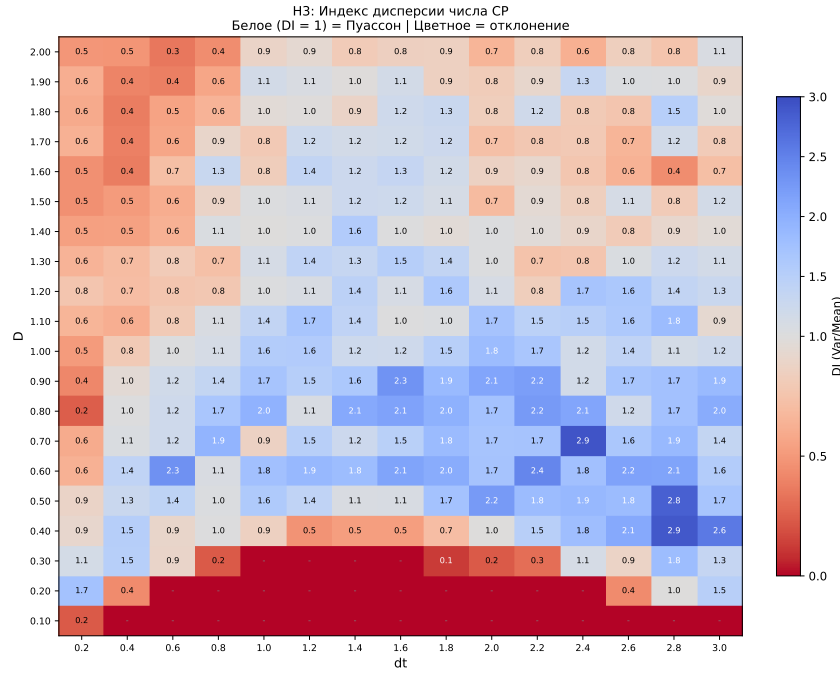


Рис. 5: Индекс дисперсии числа CP: $DI = \text{Var}/\text{Mean}$. Белые ячейки ($DI \approx 1$) – пуассоновское поведение. Цветные – отклонение: синие ($DI < 1$) – неодисперсия или мало данных, красные ($DI > 1$) – передисперсия.

6.4 Сводка гипотез

Результаты Н2 и Н3 визуализированы на рис. 6. При малом D переходов мало, распределение расстояний сильно скошено ($k \ll 1$). При большом D поток переходов стационарен ($DI \approx 1$), а распределение приближается к экспоненциальному ($k \rightarrow 1$). Зона $D = 1.0 - 1.5$ – промежуточная: переходов достаточно для статистики, но распределение еще далеко от экспоненциального.

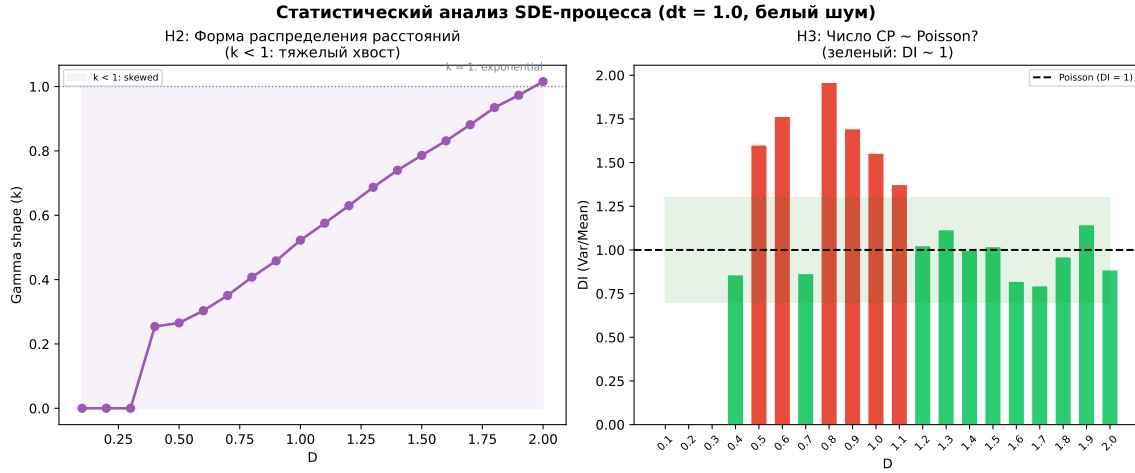


Рис. 6: Сводка статистического анализа ($dt = 1.0$, белый шум). Слева: параметр формы k гамма-распределения расстояний между CP (H2). Справа: индекс дисперсии числа CP (H3).

7 Экспериментальные результаты

7.1 Сравнение методов при $D = 1.0$ (рабочий режим)

Таблица 3 содержит результаты при $D = 1.0$, белый шум – основной режим, в котором задача содержательна.

Таблица 3: Результаты на 50 тестовых рядах, $D = 1.0$, белый шум. Показаны 8 методов с наибольшим F1. Опущены: Buishand (4 варианта, $F1 = 0.049$), Nyblom (0.049), Pettitt (0.047), GP (0.155), MDL (0.300), SWAB (0.265) и v1-модели ($F1 < 0.40$, см. раздел 3.2). σ – стандартное отклонение F1.

Метод	Прес.	Рес.	F1	σ_{F1}	ROC-AUC	PR-AUC
Transformer_v2	0.77	0.64	0.698	0.058	0.815	0.799
LSTM_v2	0.84	0.57	0.675	0.080	0.813	0.811
GRU_v2	0.91	0.51	0.650	0.073	0.847	0.850
CUSUM	0.46	0.65	0.534	0.060	0.483	0.467
CatBoost	0.92	0.33	0.479	0.038	0.733	0.788
SNHT	1.00	0.03	0.049	0.009	–	–
Chow	0.96	0.02	0.047	0.014	0.538	0.543
Pettitt	0.96	0.02	0.047	0.013	–	–

Три нейросетевых метода v2 уверенно лидируют (табл. 3). Лучший – Transformer_v2, за ним LSTM_v2 и GRU_v2; все три сопоставимы между собой, а отрыв от лучшего классического метода (CUSUM) устойчив и не объясняется случайным разбросом между рядами. Это подтверждает парный тест Уилкоксона – он сравнивает F1 каждого метода на одних и тех же 50 рядах и проверяет, систематически ли один метод лучше другого: Transformer_v2 значительно превосходит CUSUM ($p < 0.0001$), тогда как различие между Transformer_v2 и LSTM_v2 незначимо ($p = 0.094$).

Классические тесты SNHT и Chow, которые при $D = 0.5$ были безусловными лидерами (табл. 4), здесь практически не работают – recall около 3%. Причина

понятна: при ~ 200 переходах на ряд однократные тесты на однородность находят лишь один, самый выраженный разрыв. Визуально это хорошо видно на примере SDE-траектории (рис. 1): SNHT и Chow ставят по одной точке на весь ряд.

CUSUM – единственный классический метод, конкурирующий с ML по recall (0.65 – даже выше, чем у нейросетей), но precision (0.46) хромает – слишком много ложных срабатываний. ML-модели, напротив, осторожнее: GRU_v2 при precision 0.91 пропускает половину переходов, но почти не врет. Компромисс precision – recall для всех методов показан на рис. 10 (справа), а общее ранжирование по F1 – на рис. 7.

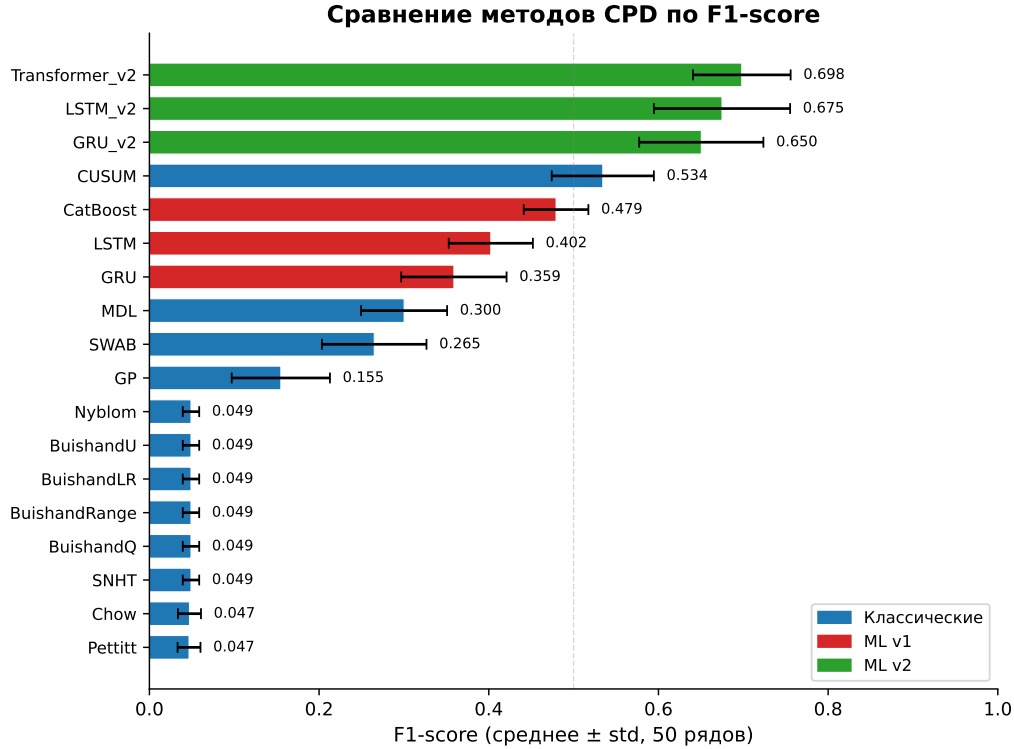


Рис. 7: F1-score всех методов при $D = 1.0$, белый шум. ML-модели v2 (зеленые) уверенно лидируют; классические тесты (синие) показывают recall $< 3\%$.

7.2 Сравнение при $D = 0.5$ (простой режим)

Результаты при $D = 0.5$ приведены в табл. 4.

Таблица 4: Результаты на 50 тестовых рядах, $D = 0.5$, белый шум.

Метод	Precision	Recall	F1	ROC-AUC	PR-AUC
SNHT	1.00	0.64	0.725	–	–
Chow	0.92	0.56	0.645	0.684	0.533
GRU_v2	0.32	0.23	0.253	0.919	0.551
CatBoost	0.36	0.21	0.243	0.650	0.211
LSTM_v2	0.24	0.21	0.207	0.866	0.424
Buishand (LR)	0.44	0.13	0.205	0.486	0.173

SNHT (F1 = 0.725) и Chow (F1 = 0.645) превосходят ML-модели с большим отрывом (рис. 10, слева). Recall SNHT = 0.64 может показаться неожиданным для теста,

который ищет ровно одну точку: при ~ 2 CP на ряд recall усредняется по рядам, и на рядах с 1 CP SNHT получает recall = 1.0, а на рядах с 3+ CP – recall падает до 0.33 и ниже. В среднем по 50 рядам это дает 0.64. При ~ 11 переходах на 10 000 шагов задача сводится к поиску единичного сдвига в почти стационарном ряде – именно то, для чего создавались тесты на однородность. ML-модели, обученные на рядах с десятками и сотнями переходов, здесь не в своей стихии.

ROC-AUC у GRU_v2 (0.919) и LSTM_v2 (0.866) при $D = 0.5$ остается высоким – модели «чувствуют» разладки, но их пороги, откалиброванные на более шумных данных, слишком агрессивны. При подборе порога отдельно под $D = 0.5$ F1 был бы существенно выше, но это требует знания D заранее, чего на практике обычно нет.

7.3 Влияние типа шума и уровня диффузии

Рис. 8 показывает F1-score ключевых методов по четырем сценариям. Видно, что Transformer_v2 стабильно лидирует при $D \geq 1.0$, а при розовом шуме ($D = 1.0$, pink) его преимущество даже усиливается: коррелированный шум затрудняет работу классических тестов, но не мешает нейросетям.

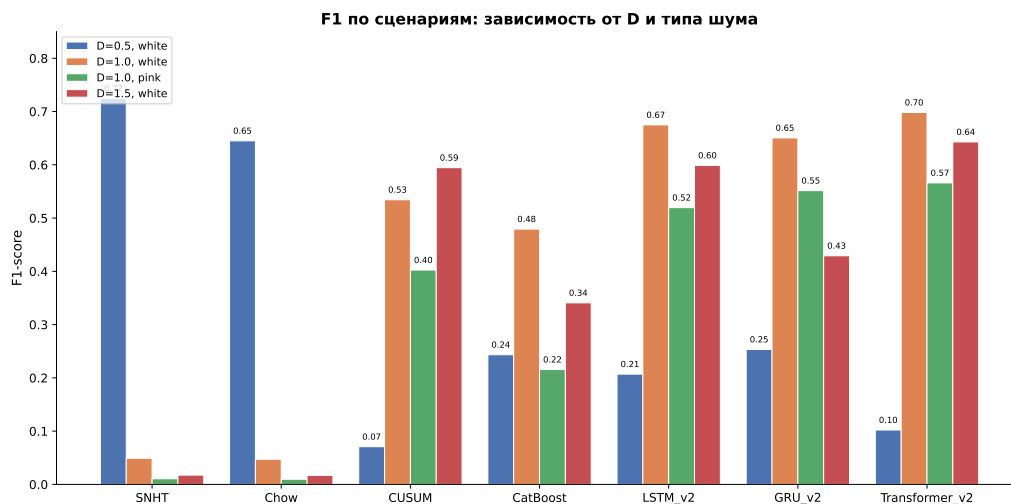


Рис. 8: F1-score ключевых методов по четырем сценариям: $D = 0.5$ (white), $D = 1.0$ (white и pink), $D = 1.5$ (white).

Другой ракурс – ROC-AUC, который оценивает качество score-функции без привязки к конкретному порогу. На рис. 9 видно, что ML-модели v2 стабильно держат ROC-AUC выше 0.80 во всех сценариях, тогда как CUSUM и CatBoost заметно просаживаются. Это говорит о том, что сами score-функции нейросетей информативны – вопрос лишь в правильном выборе порога.

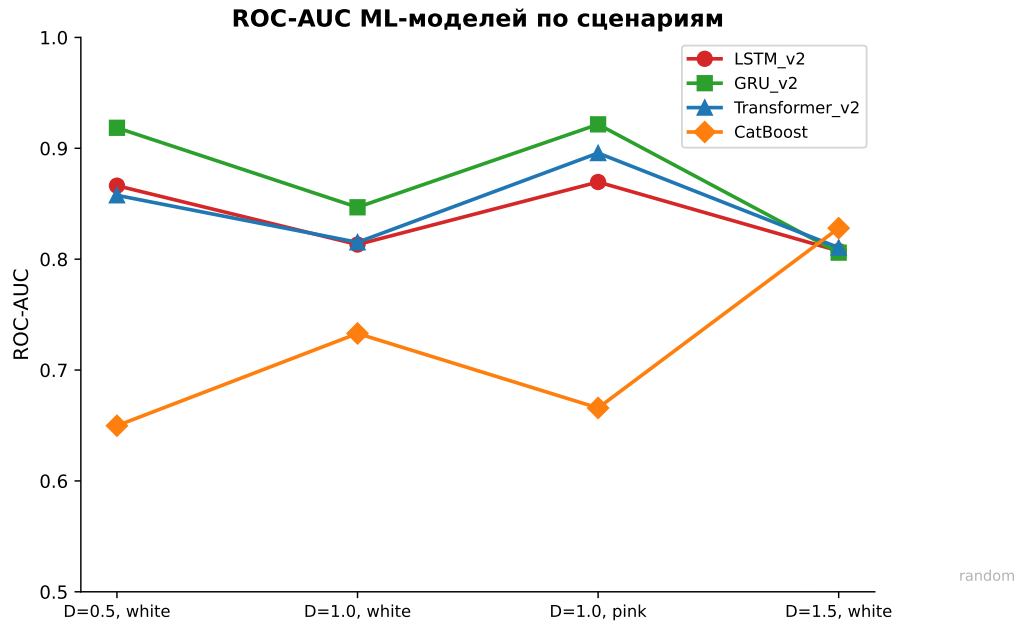


Рис. 9: ROC-AUC ML-методов по четырем сценариям. Модели v2 сохраняют высокое качество ранжирования даже при смене параметров генерации.

Компромисс precision – recall показан на рис. 10: каждый метод занимает свою нишу – от «осторожных» (высокий precision, низкий recall) до «агрессивных» (наоборот).

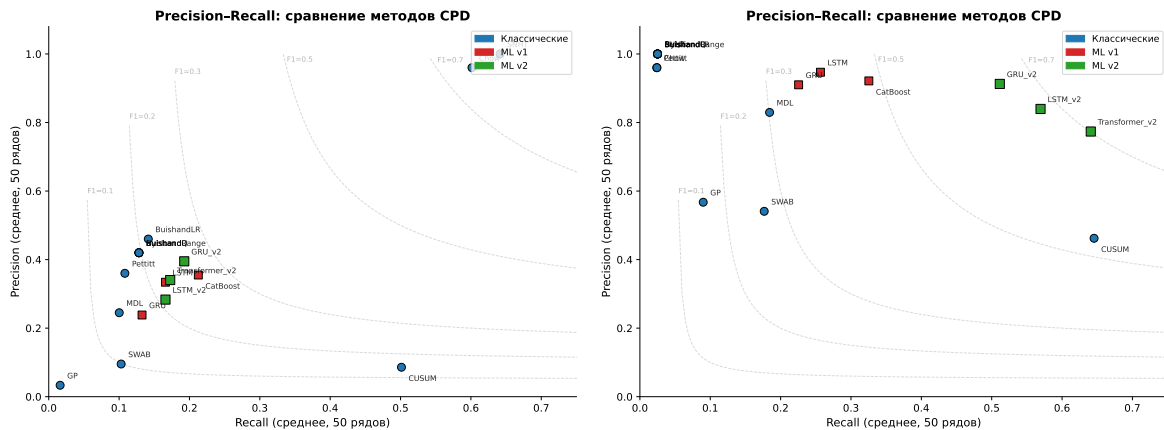


Рис. 10: Scatter-диаграмма Precision – Recall для $D = 0.5$ (слева) и $D = 1.0$ (справа). Изолинии – уровни F1.

7.4 Точность локализации

Для методов, которые детектируют разладки, важно не только «нашел/не нашел», но и «насколько точно». Средняя абсолютная ошибка локализации (MAE) показывает, на сколько шагов предсказание отстоит от истинной позиции разладки (рис. 11).

При $D = 1.0$ лучшую точность локализации показали GRU (MAE = 1.5 шага) и CatBoost (MAE = 2.3 шага) – при условии, что детекция состоялась. LSTM_v2 и Transformer_v2 дали MAE около 3.4 – 3.6. Когда ML-методы находят разладку,

они попадают с точностью до нескольких шагов – значительно лучше, чем CUSUM (MAE = 9.8), который «видит» разладки, но локализует их неточно.

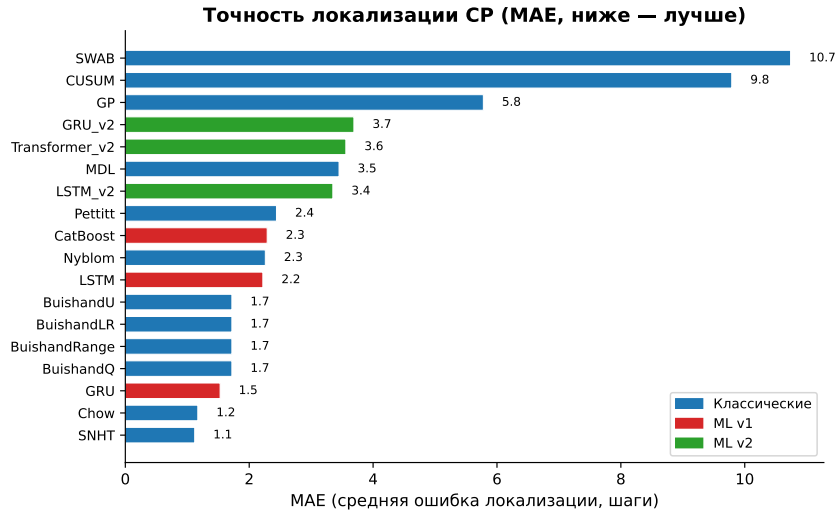


Рис. 11: Средняя абсолютная ошибка локализации (MAE) для методов с MAE < 50.

7.5 Сводка по сценариям

Таблица 5 собирает лучшие результаты по классам методов через все четыре сценария.

Таблица 5: Лучший классический и лучший ML-метод по F1 в каждом сценарии (50 тестовых рядов по 2000 шагов).

Сценарий	Лучший классич.	F1	Лучший ML	F1
$D = 0.5$, white	SNHT	0.725	GRU_v2	0.253
$D = 1.0$, white	CUSUM	0.534	Transformer_v2	0.698
$D = 1.0$, pink	CUSUM	0.402	Transformer_v2	0.566
$D = 1.5$, white	CUSUM	0.595	Transformer_v2	0.643

При $D = 0.5$ классика выигрывает с большим отрывом. При $D \geq 1.0$ ML-методы стабильно лидируют (рис. 8). CUSUM – единственный классический метод, конкурентоспособный в сложных сценариях, но уступающий Transformer_v2 на 0.05 – 0.16 по F1.

8 Интерпретация и обсуждение

8.1 Почему разные методы побеждают в разных режимах

При $D \geq 1.0$ задача – распознавание сложных паттернов: осцилляции у границы π , последовательности быстрых переходов, наложение дрефта и шума. Нейросети обучаются на примерах таких паттернов. Transformer_v2 оказался лучшим (F1 = 0.698), LSTM_v2 сопоставим (0.675), GRU_v2 чуть хуже (0.650). Разница между Transformer_v2 и LSTM_v2 статистически пограничная, а вот между тройкой v2 и остальными методами – значимая.

При $D = 0.5$ картина обратная: переходов мало, они ярко выражены, и SNHT с Chow оптимально заточены именно под это – один четкий сдвиг в стационарном ряде. ML-модели, обученные на данных с десятками переходов, «галлюцинируют» – находят лишние разладки, снижая precision. Причина не в теоретических ограничениях нейросетей, а в несоответствии обучающего и тестового режимов.

8.2 Режимный анализ

Сформулируем практические рекомендации (задача 5). Статистический анализ SDE-процесса (раздел 6, рис. 6) позволяет связать свойства данных с оптимальным методом. Ключевые индикаторы – параметр формы k гамма-распределения межпереходных расстояний и индекс дисперсии DI числа разладок:

- **Малый шум** ($D < 0.7$, $k < 0.4$): переходов мало, они хорошо выражены – применять классические тесты (SNHT, Chow). ML-модели здесь проигрывают не потому, что не «чувствуют» разладки (ROC-AUC у GRU_v2 при $D = 0.5$ составляет 0.919), а потому, что их пороги откалиброваны на более шумные данные. При повторном подборе порога под конкретный D результат будет лучше, но в этом режиме классика и так справляется.
- **Рабочий режим** ($D \in [1.0, 1.5]$, $k \approx 0.5 - 0.8$; $DI \approx 1$ при $D \geq 1.2$): разладки образуют стационарный поток, задача нетривиальна – применять ML (Transformer_v2 или LSTM_v2).
- **Сильный шум** ($D > 1.8$, $k \rightarrow 1$): время пребывания во впадине становится экспоненциальным, переходы независимы. Задача формально проще, но разладки идут так плотно, что практическая полезность детекции снижается.

Граница между «территорией классики» и «территорией ML» лежит в районе $D \approx 0.7 - 1.0$. Статистический порог ($D \leq 0.6$ из анализа перелётов, раздел 6.1) и экспериментальный ($D < 0.7$ из сравнения методов) не совпадают точно; зона $D \in [0.6, 0.7]$ – переходная.

8.3 Анализ ошибок

Средние метрики скрывают структуру ошибок. Чтобы понять, *как именно* ошибаются разные методы, посмотрим на типичный ряд при $D = 1.0$ (белый шум, в среднем ~ 40 истинных разладок):

- **Transformer_v2** предсказывает 34 точки, из которых ~ 8 ложных и ~ 15 пропущенных – сбалансированный профиль ошибок.
- **CUSUM** выдаёт 57 предсказаний: находит больше всех (recall 0.65), но 31 из них ложные – «шумный» детектор.
- **GRU_v2** осторожнее всех: 23 предсказания, из них ложных всего 2, зато пропущено 20 – «тихий» детектор.

Здесь стоит оговориться: F1 неявно предполагает равную цену ложной тревоги и пропуска. На практике это не всегда так. Однако у ML-моделей есть преимущество: непрерывный score позволяет сдвигать порог θ и перемещаться вдоль всей кривой

precision – recall. Если пропустить разладку дорого – достаточно снизить порог, и recall вырастет за счёт precision. Высокий ROC-AUC у Transformer_v2 (0.815) гарантирует, что при любом выбранном компромиссе он будет лучше CUSUM, у которого ROC-AUC всего 0.483. Классические single-CP тесты (SNHT, Chow) такой гибкости лишены – они выдают одну точку и не имеют настраиваемого порога.

В худших случаях Transformer_v2 проседает до $F1 \approx 0.56 - 0.60$. Это происходит на рядах, где процесс активно осциллирует вблизи границы π , не совершая «настоящего» перехода – модель принимает эти колебания за разладки. CUSUM страдает от той же проблемы, но сильнее, потому что реагирует на любой сдвиг среднего, а SDE-процесс вблизи π дает ложные сдвиги постоянно.

8.4 Вычислительная сложность

На наших рядах (2000 шагов, Kaggle T4 GPU) инференс всех методов укладывается в секунды: классические тесты – доли секунды (кроме SNHT: ~ 4 с (полный перебор позиций разделения)), CatBoost и GRU – доли секунды, Transformer_v2 – около 0.8 секунды. Обучение – другой вопрос: CatBoost обучается за 1 – 2 минуты на CPU, рекуррентные сети – за 10 – 30 минут на GPU, Transformer_v2 – около часа. Если веса уже есть, все ML-методы работают так же быстро, как классика. Таким образом, выбор между классикой и ML определяется не вычислительными затратами, а свойствами данных (см. раздел 8.3).

8.5 Ограничения

Работа проведена на синтетических данных одного типа (SDE с sin-потенциалом). Это сознательный выбор, дающий полный контроль над параметрами, но ограничивающий генерализацию: мы не знаем, как методы поведут себя на SDE с другим потенциалом (например, $V(x) = x^4 - x^2$) или на многомерных процессах. Реальные ряды устроены иначе, и перенос выводов требует проверки.

Для контекста: на стандартных кусочно-стационарных бенчмарках нейросетевые методы CPD достигают F1 порядка 0.80 – 0.90 [17, 18]. На наших SDE-данных потолок ниже – 0.698 при $D = 1.0$. Это отражает повышенную сложность задачи: разладки здесь не скачки между двумя уровнями, а переходы через потенциальный барьер, маскируемые шумом.

Отдельно стоит отметить низкий F1 у Transformer_v2 при $D = 0.5$ (0.102 – худший среди ML). Это не ограничение архитектуры, а следствие обучающей выборки: модель тренировалась на рядах с десятками переходов и при $D = 0.5$ «галлюцинирует» лишние (precision 0.06). GRU_v2 в том же режиме показывает ROC-AUC = 0.919 – нейросети *способны* детектировать разладки при малом шуме, но для этого нужна повторный подбор порога или переобучение на соответствующем D . Если D известен заранее, специализированная ML-модель, вероятно, догонит классику; если нет – SNHT надёжнее.

9 Заключение

Мы получили два конкретных правила. Первое: при $D < 0.7$ нейросети в универсальной конфигурации (обученные на широком диапазоне D) уступают простому

SNHT; для конкуренции с классикой им нужен повторный подбор порога или переобучение под конкретный режим. Второе: при $D \geq 1.0$ Transformer_v2 выигрывает у лучшего классического метода (CUSUM) на 0.16 по F1, и это подтверждается парным тестом Уилкоксона ($p < 0.0001$). Граница между этими режимами – район $D \approx 0.7 - 1.0$ – определена экспериментально и подтверждена статистическим анализом SDE-процесса.

Три наблюдения, которые мы не ожидали. Во-первых, самый большой прирост качества ML-моделей дала не архитектура и не объём данных, а приведение в соответствие статистик обучающей и тестовой выборок ($v1 \rightarrow v2$, рост F1 на 0.3 – 0.5). Во-вторых, CUSUM – метод 1954 года – при $D = 1.0$ показал recall выше, чем любая нейросеть. Он проиграл по F1 только из-за precision, а не из-за чувствительности. В-третьих, ML-модели «чувствуют» разладки даже в режиме, где проигрывают по F1: ROC-AUC у GRU_v2 при $D = 0.5$ составляет 0.919, что означает существование порога, при котором модель работает хорошо – просто текущий порог не тот. Это важно практически: имея непрерывный score, ML-модель можно настроить на любой компромисс precision – recall, тогда как классические single-CP тесты такой гибкости лишены.

На SDE-данных лучший достигнутый F1 составляет 0.698 (против 0.80 – 0.90 на стандартных бенчмарках). Разрыв отражает повышенную сложность задачи: разладки здесь не скачки между двумя уровнями, а переходы через потенциальный барьер, маскируемые шумом. Статистический анализ процесса (параметр формы k межпереходных расстояний, индекс дисперсии числа CP) показал, при каких параметрах задача тривиальна, а при каких содержательна. Это может быть полезно при проектировании SDE-генераторов для задач CPD.

Воспроизводимость. Все эксперименты воспроизводимы: фиксированные seeds для генерации (train 0 – 99, val 5000 – 5019, test 1000 – 1049), единый конфиг (configs/base.yaml), предобученные веса в репозитории. Полный eval запускается одной командой (python eval_multi.py - n-series 50) и занимает несколько минут при наличии весов.

Ход работы. Проект шёл не линейно. Первые эксперименты проводились при $D = 0.5$, где классические тесты уверенно лидируют, и ранний вывод был: нейросети не нужны. Лишь после перехода к $D = 1.0$ картина изменилась на противоположную. Аналогично, наибольший прирост качества ML-моделей дала не смена архитектуры (разница между LSTM, GRU и Transformer составила 0.02–0.05 по F1), а исправление состава обучающей выборки ($v1 \rightarrow v2$, рост F1 в 1.7–1.8 раза). Оба факта — иллюстрация центрального тезиса работы: результат сравнения методов определяется не столько алгоритмами, сколько данными, на которых они обучаются и тестируются.

Что дальше. Два направления. Первое – проверить рекомендации на реальных данных: финансовых рядах и физических экспериментах, где переключения между устойчивыми состояниями встречаются в явном виде. Второе – обучить ML-модели отдельно для каждого режима D (или сделать D входным параметром модели), чтобы устранить главный источник ошибок – несоответствие между обучающей и тестовой статистиками.

Список литературы

- [1] Page E.S. Continuous inspection schemes. *Biometrika*, 41(1/2): 100 – 115, 1954.

- [2] Pettitt A.N. A Non-Parametric Approach to the Change-Point Problem. *J. R. Stat. Soc. C*, 28(2): 126 – 135, 1979.
- [3] Yao Y.C. Estimating the number of change-points via Schwarz’ criterion. *Statistics & Probability Letters*, 6(3): 181 – 189, 1988.
- [4] Basseville M., Nikiforov I.V. Detection of abrupt changes: theory and application. Prentice Hall, 1993.
- [5] Hochreiter S., Schmidhuber J. Long Short-Term Memory. *Neural Computation*, 9(8): 1735 – 1780, 1997.
- [6] Keogh E., Chu S., Hart D., Pazzani M. Segmenting time series: A survey and novel approach. In *Data Mining in Time Series Databases*, 1 – 21. World Scientific, 2004.
- [7] Takeuchi J., Yamanishi K. A unifying framework for detecting outliers and change points from time series. *IEEE Trans. Knowl. Data Eng.*, 18(4): 482 – 492, 2006.
- [8] Adams R.P., MacKay D.J.C. Bayesian Online Changepoint Detection. *arXiv:0710.3742*, 2007.
- [9] Sugiyama M., Suzuki T., Nakajima S. et al. Direct importance estimation for covariate shift adaptation. *Ann. Inst. Stat. Math.*, 60(4): 699 – 746, 2008.
- [10] Saatçi Y., Turner R.D., Rasmussen C.E. Gaussian process change point models. In *ICML*, 927 – 934, 2010.
- [11] Killick R., Fearnhead P., Eckley I.A. Optimal detection of changepoints with a linear computational cost. *J. Amer. Stat. Assoc.*, 107(500): 1590 – 1598, 2012.
- [12] Liu S., Yamada M., Collier N., Sugiyama M. Change-point detection in time-series data by relative density-ratio estimation. *Neural Networks*, 43: 72 – 83, 2013.
- [13] Cho K., van Merriënboer B., Gulcehre C. et al. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv:1406.1078*, 2014.
- [14] Aminikhanghahi S., Cook D.J. A survey of methods for time series change point detection. *Knowl. Inf. Syst.*, 51(2): 339 – 367, 2017.
- [15] Vaswani A., Shazeer N., Parmar N. et al. Attention Is All You Need. In *NeurIPS 30*, 5998 – 6008, 2017.
- [16] Prokhorenkova L., Gusev G., Vorobev A. et al. CatBoost: unbiased boosting with categorical features. In *NeurIPS 31*, 6638 – 6648, 2018.
- [17] Truong C., Oudre L., Vayatis N. Selective review of offline change point detection methods. *Signal Processing*, 167: 107299, 2020.
- [18] Hushchyn M., Arzymatov K., Derkach D. Online Neural Networks for Change-Point Detection. *arXiv:2010.01388*, 2020.
- [19] Atashgahi Z., Mocanu D.C., Veldhuis R., Pechenizkiy M. Memory-free Online Change-point Detection: A Novel Neural Network Approach. *arXiv:2207.03932*, 2022.
- [20] Xu R., Song Z., Wu J., Wang C., Zhou S. Change-point detection with deep learning: A review. *Front. Eng. Manag.*, 12(1): 154 – 176, 2025.